IPB University
Bogor Indonesia

DIKTISAINTEK BERDAMPAK

MK Analisis Algoritme

Pertemuan 6: Algoritme Pencarian (*searching*)



Tim Pengajar
MK Analisis Algoritme

Tujuan Pembelajaran

- Mahasiswa mampu menjelaskan beberapa jenis algoritme pencarian yang umum digunakan (numerik, terurut, linear/spasial, string)
- Mahasiswa mampu menganalisis kompleksitas dari algoritme-algoritme pencarian tersebut
- Mahasiswa mampu menjelaskan keunggulan dan kelemahan dari masing-masing algoritme pencarian

Review Algoritme Pencarian

- Algoritme pencarian:

diberikan sebuah array $A[1 \ldots n]$ dan sebuah nilai a, carilah sebuah indeks *i* (pertama) sedemikian rupa sehingga $A[i] = a$

- Ada dua buah algoritme yang paling sering digunakan:
  - Linear Search
  - Binary Search

Linear vs Binary Search

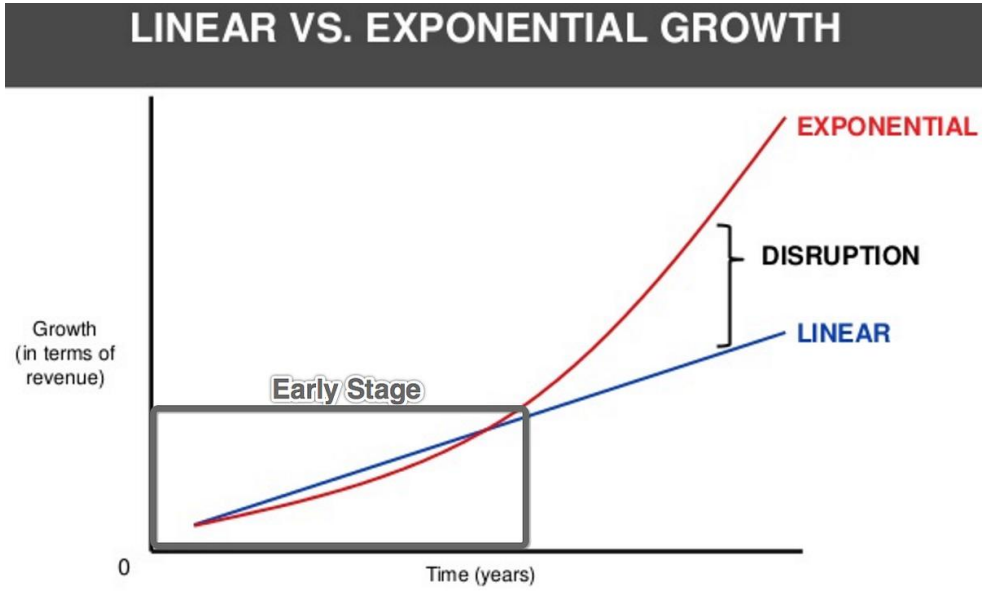
Berikut adalah table perbandingan antara *linear* dan *binary search*

No	Properti	Linear Search	Binary Search
1	Prasyarat Data	Tidak ada	Data terurut (sorted)
2	*Worst case* complexity	O(N)	O(log N)
3	Metode pencarian	Sekuensial	Divide & Conquer
4	Cocok untuk	Data kecil tidak terurut	Data besar terurut

IPB University
Bogor Indonesia

Interpolation Search

- **Ide dasar:** memperbaiki kompleksitas dari *binary search* dengan cara **memberikan estimasi (tebakan)** yang lebih tepat terhadap **posisi** nilai yang dicari, menggunakan **interpolasi linear**
- Dengan mengestimasi posisi yang lebih tepat, diharapkan kita mendapatkan nilai yang dicari **lebih cepat**
- Kita mengasumsikan data kita **terurut** dengan **distribusi** yang merata



IPB University
Bogor Indonesia

Interpolasi Linear

- Interpolasi linear berarti menghitung/mengestimasi nilai sebuah elemen dalam *array* dengan mengasumsikan bahwa *array* tersebut terurut dan nilai-nilai elemennya mendekati **barisan** aritmetika
  - Misal: 3, 7, 11, 15, ..., ada di **posisi berapakah** nilai 1000 di barisan tsb?
- Jika nilai-nilai sebuah array **mendekati** sebuah barisan aritmetika, maka kita dapat mengestimasinya dengan lebih akurat. Hal ini serupa dengan mengira-ngira posisi nomor telepon seseorang di buku telpon



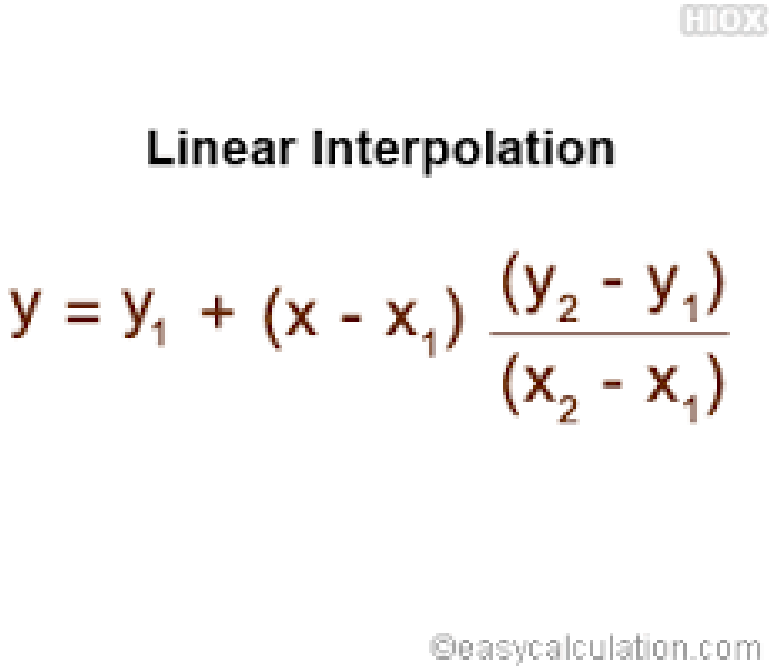
IPB University
Bogor Indonesia

Interpolasi Linear

- Diberikan dua buah titik

(x_1, y_1) dan (x_2, y_2)

Kita dapat menghitung semua data yang ada pada garis yang menghubungkan kedua titik tersebut menggunakan formula berikut



IPB University
Bogor Indonesia

Interpolation Search

- Interpolation search menambahkan tahapan **interpolasi** untuk mengestimasi posisi dari nilai yang akan dicari
- Berpotensi mengurangi langkah pencarian apabila data **terdistribusi** secara linear

Interpolation Search

$$pos = lo + \left[\frac{(target - arr[lo]) * (hi - lo)}{(arr[hi] - arr[lo])} \right]$$

IPB University
Bogor Indonesia

Interpolation search

```
1  INTERPOLATION-SEARCH(A[1..n], x):
2      L ← 1
3      R ← n
4      while L ≤ R and x ≥ A[L] and x ≤ A[R]:
5          // avoid division by zero (implied by guard above if A[L] ≠ A[R])
6          pos ← L + ⌊ (x - A[L]) * (R - L) / (A[R] - A[L]) ⌋
7
8          if A[pos] = x:
9              return pos
10         else if A[pos] < x:
11             L ← pos + 1
12         else:
13             R ← pos - 1
14     return NOT-FOUND
```

IPB University
Bogor Indonesia

Interpolation Search

- **Best case** : **O(1)**: langsung ketemu
- **Average case: O(log log n)**
  - pada tebakan ke-*k*, ukuran rentang tersisa adalah $r_k = n^{\left\{\frac{1}{2}\right\}^k}$. Pada saat ditemukan, maka nilai $r_k \approx 1$ yang berarti $2^k \approx log\ n$ atau $k \approx log\ log\ n$
- **Worst case: O(n)**
  - Jika data **sangat** tidak uniform (merata), maka dalam kasus terburuk, kita hanya mengeliminasi **satu** nilai pada setiap langkah, sehingga $k \approx n$

IPB University
Bogor Indonesia

Exponential Search

- Apabila ukuran array **sangat besar (tidak terbatas)**, kita mungkin dapat melakukan pencarian lebih cepat dengan menggunakan variasi dari *binary search* dimana **jarak lompatan** kita diperbesar, sampai ditemukan nilai yang sesuai
- Exponential search terbagi menjadi dua:
  - Tahap pertama: mencari **rentang** indeks nilai sedemikian rupa sehingga nilai yang dicari < 2^j dimana *j* adalah sebuah eksponen
  - Tahap kedua: melakukan binary search pada semua array antara nilai pertama dan 2^j

Exponential Search

```
1  EXPONENTIAL-SEARCH(A[1..n], x):
2      if n = 0: return NOT-FOUND
3      if A[1] = x: return 1
4
5      i ← 1
6      // gandakan sampai melewati x atau ujung array
7      while i ≤ n and A[i] < x:
8          i ← i * 2
9
10     L ← (i / 2) + 1
11     R ← min(i, n)
12     return BINARY-SEARCH(A, x, L, R)
```

Exponential Search

- Pada dasarnya, kompleksitas dari exponential search tidak berbeda secara asimtotik dari *binary search*, namun dalam prakteknya, jumlah langkah yang diperlukan dapat berkurang
  - Best case: O(1)
  - Average case: O(log n)
  - Worst case: O(log n)

IPB University
Bogor Indonesia

Pencarian dalam ruang 2D (matriks)

- Diberikan matriks (array 2D) $A[m \times n]$ dan nilai yang akan dicari a, tentukan indeks *i* dan *j* sedemikian rupa sehingga $A[i][j] = a$

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ \vdots \\ m \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & \cdots & n \end{array} \\ \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \end{array}$$



a

IPB University
Bogor Indonesia

Pencarian dalam ruang 2D (matriks)

- Ada 3 kemungkinan distribusi data:
  - Kondisi 1: tidak ada urutan pada data (acak)
  - Kondisi 2: tiap baris terurut (misalnya menaik), tapi antarbaris tidak terurut
  - Kondisi 3: baris dan kolom terurut (monoton 2D)
- Setiap kondisi memiliki algoritme yang paling sesuai

IPB University
Bogor Indonesia

Kondisi tidak terurut

```
1  SEARCH-2D-UNSTRUCTURED(M, x):
2      for i ← 1..n:
3          for j ← 1..m:
4              if M[i][j] = x:
5                  return (i, j)      // untuk “locate one”
6      return NOT-FOUND
7
```

- Berapa kompleksitasnya?

IPB University
Bogor Indonesia

Kondisi terurut perbaris

- Setiap baris berada pada kondisi terurut → dapat menerapkan *binary search* pada setiap baris
- Antarbaris tidak ada relasi, sehingga tidak dapat menyimpulkan apa-apa → semua baris harus dicari (sampai ketemu)
- Bagaimana algoritme persisnya?
- Berapakah kompleksitas dari algoritme tersebut?

IPB University
Bogor Indonesia

Kondisi terurut 2D (monoton)

- Struktur data:
  - setiap baris terurut (monoton)
  - setiap kolom terurut (monoton)
- Intuisi algoritme:
  - Jika kita membandingkan sebuah nilai *x* dengan A[i][j], maka kita dapat mengeliminasi **semua baris** di atas/bawah *i* serta **semua kolom** di kanan/kiri *j* sesuai dengan hasil perbandingan nilai *x* dengan A[i][j]
- Hal ini memunculkan sebuah pola *tangga* (staircase)

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

IPB University
Bogor Indonesia

Staircase Search

```
1  STAIRCASE-SEARCH(M[1..n][1..m], x):      // rows & cols non-decreasing
2      i ← 1; j ← m                         // start at top-right
3      while i ≤ n and j ≥ 1:
4          if M[i][j] = x: return (i, j)
5          else if M[i][j] > x: j ← j - 1   // eliminate column j
6          else:                i ← i + 1   // eliminate row i
7      return NOT-FOUND
```

- Berapakah kompleksitas waktunya?

IPB University
Bogor Indonesia

Latihan

- Tentukan algoritme apa yang terbaik untuk melakukan pencarian dalam sebuah matriks $A[m \times n]$ dimana

$$A[i][n] \leq A[i+1][1], 1 \leq i \leq m-1$$

- Tentukan kompleksitas waktu algoritme tersebut!



IPB University

Bogor Indonesia

Inspiring Innovation with Integrity

in Agriculture, Ocean and Biosciences for a Sustainable World